

Grado en Ingeniería Informática.
MTPA. Universidad Europea Miguel de Cervantes
Profesor: Javier Garrido Díaz (jgarrido@uemc.es)
22 de abril de 2026

Practica Final

Métodos y Técnicas de Programación Avanzada

Practica Final	1
Control de Versiones	2
Introducción	3
Objetivos de la práctica	3
Idea Principal	3
Requisitos Funcionales	3
Salones	4
Mensajes Privados	4
Registro. Login	5
Heartbeat	5
Notificaciones	5
Observabilidad	5
Calidad	5
Implementación de Servidor	6
Persistencia de información	6
Hitos y Entrega	6
Documentación a Entregar	7
Consejos de Implementación	7

Control de Versiones

Fecha	Version	Comentarios
22 de Abril 2026	1.0	Version Inicial

Introducción

El siguiente documento posee la definición de la practica final de la asignatura de Métodos y Técnicas de Programación Avanzada del curso 2025/2026.

Esta definición puede variar, por lo que es necesario revisar el historial de versiones.

Objetivos de la práctica

Se pretende que con este practica el alumno adquiera destrezas en la programacion de interfaces de usuario, programación con Sockets, trabajar con sistemas multiprocesamiento así como la concepción y construcción de pruebas unitarias. Se pretende además que el alumno sea capaz de generar un protocolo de aplicación de Capa 7 adecuado a las necesidades que puedan aparecer en el enunciado.

En este caso la práctica consistirá en desarrollar un sistema de mensajería tipo “**Slack o Discord**” usando para ello sistemas conectados.

Idea Principal

A grandes rasgos, la implementación general consistirá en:

- El alumno debe desarrollar un sistema de mensajería centralizado en un servidor.
- Clientes de interfaz de usuario con Java AWT y/o Java Swing.
- Se deberá crear un protocolo propio de aplicación de Caja 7, para aquellos comandos que considere el alumno.
- El servidor NO ofrecerá una interfaz de usuario enriquecida, pero ofrecerá funcionalidades en modo “administración”.
- El sistema deberá manejar concurrencia entre varios clientes, al menos 10.
- NO está permitido el uso de Frameworks de alto nivel como Java RMI, Spring, WebSocker
- Se de usar exclusivamente Socket, ServerSocket, Thread y Runnable. Librerías standard de Java.

Requisitos Funcionales

Se debe desarrollar una aplicación de mensajería distribuida de tal forma que cada usuario pueda enviar mensajes a salones de Chat. Es decir: El sistema base es un grupo de chats/hilos que permite enviar mensajes en tiempo real al salón para que el resto de usuarios puedan interactuar entre sí. Adicional a esto, el sistema debe permitir el envío de mensajes privados entre los usuarios.

La aplicación se debe basarse en una aplicación centralizada “Cliente/Servidor” en la que “por el servidor” deben pasar todas las operaciones de mensajes. Todos los clientes se comunican a través del servidor. NO Hay comunicación de Cliente a Cliente.

Salones

En esencia un salón es un lugar donde se escriben mensajes que tienen que ver con el nombre del salón; seria similar a “un Topic o Tema”, por el cual se envían mensajes del Tópico. Eso no quita para que un usuario incluya mensajes que no son del tema. Son solo normas que un usuario puede seguir o no. Cuando un usuario entra a un salón se le muestra los mensajes sólo del último día; sin embargo, es necesario implementar “algo” para que el usuario pueda cargar mensajes “de días anteriores”.

Por norma general, el contenido del salón solo podrá ser texto, pero deberá mostrarse el usuario que envía el mensaje, la fecha y hora y el contenido del mensaje. Este contenido del mensaje estará limitado a 190 caracteres. Es decir, un usuario solo podrá enviar un mensaje al salón inferior o igual a 190 caracteres.

La aplicación deberá empezar con una serie de salones predefinidos, siendo estos los siguientes:

- Salón de IA.
- Salón de Deportes
- Salón Therian
- Salón Manga
- Salón UEMC

Mensajes Privados

Un usuario podrá enviar un mensaje privado a otro. Esto permitirá que la conversación entre esos dos usuarios es privada entre ellos y no se envía el mensaje a ningún salón. Además, la conversación desaparece una vez que alguno de los dos usuarios cierra la ventana privada.

Para que se pueda producir el envío de mensajes privados, el usuario que inicia la conversación privada, debe conocer el usuario con el que quiere conversar, además de estar conectado en ese momento. Por lo tanto, no es posible enviar un mensaje privado a alguien que no esta conectado.

Registro. Login

Para que un usuario pueda comunicarse entre los distintos salones, debe registrarse con un nombre de usuario único. Además, cuando se registra, se le aporta una clave de acceso, que es generada por el servidor. Esta clave debe ser *autonumérica* y se le debe enviar al usuario cuando se registra. Es importante destacar que el usuario debe ser único, así como la clave asignada, que no puede repetirse; es decir: debe ser única.

El Login de usuario se realizará por lo tanto con el nombre de usuario y la clave que se le ha asignado al usuario. Cuando el usuario haga login, lo mas natural es mostrarle la lista de salones que existen en el servidor o abrirle ventanas por cada uno de los últimos salones a los que ingresó la ultima vez.

Heartbeat

Se necesita implementar un sistema de Heartbeats. Un heartbeat es un ping o mensaje que se manda cada X minutos indicando que el usuario sigue conectado al sistema y funcionando.

Notificaciones

Se debe generar una notificación a los usuarios cuando un usuario nuevo entra en el salón, además de mostrarse como usuario del salón. Esta implementación podría realizarse a través de un mensaje “automático” en el mismo salón, o a través de un PopUp en la aplicación de cada cliente.

Observabilidad

Se deberá generar en el lado del cliente así como en el servidor, logs de la aplicación, indicando cualquier operación que pueda suceder: errores, ejecuciones correctas, etc.

Calidad

Es necesario desarrollar pruebas unitarias para el servidor, con cobertura de al menos el 30% del código.

Implementación de Servidor

El servidor será una aplicación de consola, por el que pasa toda la información / operación. Tendrá la posibilidad de:

- Dejar de aceptar más clientes
- Parar la mensajería entre clientes, para labores de mantenimiento
- Mostrar información sobre:
 - El número de usuarios conectados en el instante.
 - El número de usuarios por salón activos en ese momento
 - El numero de mensajes enviados en cada salón.

Persistencia de información

Se debe mantener almacenada de manera persistente la siguiente información:

- La lista de usuarios, con la información relevante de cada uno de ellos.
- La información relativa a los salones
- Cualquier otra información que se considere necesaria

Hitos y Entrega

La implementación de la practica se realizará entre una o dos personas máximo. Dichos grupos se deben comunicar al profesor con la mayor brevedad posible (máximo 1 de Mayo de 2026). La implementación individual debe estar justificada.

Se realizarán dos Hitos previos a la entrega definitiva. Se deben realizar las entregas de cada Hito para poder realizar la entrega final. No es posible entregar la practica sin haber realizado las entregas de Hitos.

- Hito 1: 4 de Mayo. Se deberá entregar el protocolo definido tanto para cliente como para servidor..
- Hito 2. 22 de Mayo. Implementación del Servidor.

La entrega definitiva se realizará el **2 de Junio de 2026 a través del portal Moodle**. El periodo de defensa -en el caso de existir- se comunicará con antelación.

Será necesario que el grupo de alumnos usen **Git o Subversión** como repositorio de código fuente siendo la **rama master o trunk** la que se evaluará en la fecha anteriormente indicada. Cada uno de los alumnos deben crearse un usuario y crear un repositorio para

albergar la practica (un único repositorio por grupo de práctica). Seguidamente deberán enviar una invitación para que el profesor aparezca como colaborador en dicho repositorio (se envía una invitación al email jgarrido@uemc.es). Aunque no es necesario, es recomendable trabajar en este repositorio con la metodología de ramas vista en Subversion o la metodología GitFlow

Documentación a Entregar

Se deberá entregar junto con el código fuente:

- Todos los ficheros necesarios para la correcta ejecución de la practica,
- Ficheros de persistencia.
- Documento con el protocolo de comunicaciones elegido.
- Diagrama de clases explicando las más relevantes de la aplicación.
- Documentación implementada a través de JavaDoc.

Consejos de Implementación

- Cuidar la presentación del usuario.
- Implementar siempre que se posible, multiprocesamiento.
- Implementar el protocolo correspondiente.
- Cuidar la calidad del código siempre es importante. Es recomendable usar las técnicas de *clean code* propuestas en la asignatura.